

Building a Sovereign Defense: An Architectural Blueprint for a Fully Free, AI-Powered Intrusion Detection System

The objective of this research report is to provide a comprehensive and insightful investigation into the design, development, and operationalization of a fully free, artificial intelligence (AI)-based intrusion detection and cybersecurity monitoring system. The central constraint of this investigation is the exclusive use of freely available tools, frameworks, datasets, and methodologies, thereby excluding any proprietary or commercially licensed solutions. This report deconstructs the complex task of building such a system into its constituent parts, examining the technical architecture, data strategy, implementation challenges, operational workflows, and potential future enhancements. It synthesizes information from academic papers, technical documentation, and community guides to construct a detailed blueprint for a robust, scalable, and extensible security platform built entirely on open-source principles. The analysis focuses on identifying best-in-class free components and demonstrating how they can be integrated to create a layered defense capable of detecting modern cyber threats in diverse environments, including enterprise networks, cloud infrastructures, and Internet of Things (IoT) deployments.

Architectural Blueprint: Integrating Open-Source Components into a Cohesive Security Stack

The successful construction of a powerful, AI-driven intrusion detection system under a "100% free" constraint necessitates a modular, layered architecture rather than a monolithic approach ³⁸. No single tool can provide comprehensive coverage against the spectrum of modern cyber threats; instead, the strength of the system lies in the strategic orchestration of specialized, complementary open-source components. The proposed architecture must be designed for modularity, allowing individual elements to be upgraded or replaced without requiring a complete overhaul of the entire platform. This blueprint outlines a reference architecture composed of distinct layers: a data collection

and ingestion layer, a real-time stream processing engine, an AI model deployment and serving layer, and a final layer for orchestration, correlation, and visualization. Each layer is populated with specific, high-quality open-source tools identified as best-in-class for their respective functions, forming a cohesive and powerful security stack.

The foundational layer of the architecture is dedicated to data collection and ingestion, responsible for gathering raw security telemetry from across the digital environment. This involves capturing network traffic, monitoring endpoints, scanning for vulnerabilities, and ingesting threat intelligence. For network visibility, two primary tools are essential: Zeek and Suricata. Zeek, formerly known as Bro, is consistently recommended as a premier open-source network analysis framework ^{11 38}. Unlike traditional packet-capture tools, Zeek passively monitors network traffic at a tap or SPAN port and generates rich, structured log files for every connection, DNS query, HTTP request, SSL certificate exchange, and authentication event ³⁸. These logs provide deep context that is invaluable for subsequent analysis and detection engineering, though Zeek itself does not generate alerts by default ³⁸. Complementing Zeek's deep inspection capabilities is Suricata, a powerful open-source Network-based Intrusion Detection System (NIDS) and Intrusion Prevention System (IPS) ^{11 31}. Suricata analyzes network packets in real-time against a set of rules to identify and flag known malicious patterns, providing direct, actionable alerts ³⁷. The integration of Suricata with Wazuh allows for these network alerts to be fed directly into the central security platform for correlation with other events ^{28 30}. For endpoint visibility, the Wazuh platform serves a dual role. It acts as a comprehensive Host-based Intrusion Detection System (HIDS), monitoring critical file integrity changes, unauthorized process executions, and suspicious system calls ^{11 38}. Furthermore, it functions as a centralized manager and aggregator, collecting logs from agents deployed across endpoints. To achieve granular visibility into Windows systems, which is crucial for modern security operations, deploying Sysmon (System Monitor) is a critical enhancement ³⁸. Sysmon is a free service from Microsoft that provides detailed event logging for process creation, network connections, and registry modifications, which the Wazuh agent can then collect and forward to the Wazuh manager ³⁸. Proactive threat identification requires a vulnerability management component, for which OpenVAS, particularly its community edition, is the go-to open-source solution ³⁸. Finally, to enrich the system with external threat knowledge, the Malware Information Sharing Platform (MISP) is indispensable. MISP is an open-source platform designed for gathering, storing, sharing, and correlating Indicators of Compromise (IoCs), allowing the IDS to contextualize internal alerts with global threat intelligence ^{16 18}.

Once security data is collected, it must be processed in real-time to enable immediate threat detection and response. This forms the core analytics engine of the system. The

provided sources present a detailed comparison of open-source stream processing frameworks, highlighting a clear trade-off between complexity and performance. Apache Kafka emerges as the dominant choice for the transport or message-broker layer ^{2 6}. As a distributed event streaming platform maintained by the Apache Software Foundation, Kafka is engineered for high-throughput communication and can handle millions of events per second with sub-millisecond latency ^{6 7}. Its key features, such as topic partitioning and consumer groups, make it highly scalable and resilient ². However, it is crucial to understand that Kafka functions primarily as a transport layer; it does not perform any intrinsic data processing but serves as the backbone for moving data between producers (like Wazuh agents) and consumers (like the processing engine) ⁶. For the actual real-time analytics, several powerful engines are available. Apache Flink is presented as the most sophisticated option for low-latency, stateful computations over unbounded data streams ^{2 6}. Its "stream-first" design, native support for event-time processing, and powerful windowing API make it exceptionally well-suited for complex cybersecurity use cases, such as correlating events over time to detect multi-stage attacks or anomalous user behavior ^{6 7}. In contrast, Apache Spark's Structured Streaming module offers a unified programming model for both batch and stream processing, which can be advantageous if the same team manages both data science pipelines and real-time monitoring ^{6 8}. Its tight integration with Spark's MLlib library also makes it a strong candidate for machine learning-centric applications ². The primary drawback of Spark's micro-batch model is higher latency compared to more specialized engines like Flink, typically in the range of seconds ². Redpanda presents itself as a modern, operationally simpler alternative to Kafka, written in C++ for improved performance and a smaller hardware footprint ^{2 3}. As a Kafka-compatible streaming platform, it allows existing tools and clients to be used without code changes, simplifying deployment ³. While it offers significantly lower latency than Kafka, its ecosystem and community are less mature ². Given the advanced analytical needs of an AI-driven IDS, a combination of Apache Kafka for high-throughput data transport and Apache Flink for stateful, low-latency processing represents a state-of-the-art, free architecture.

After the AI models are trained, they require a production-grade serving environment to make predictions on new, incoming data. NVIDIA Triton Inference Server is repeatedly cited as a key open-source component for this purpose ^{14 58}. It is a flexible inference serving platform that enables teams to deploy models from multiple popular frameworks, including PyTorch, TensorFlow, ONNX, and XGBoost, in a unified manner ^{58 61}. Triton handles the complexities of model loading, batching requests for efficient GPU utilization, and managing model versioning, making it a critical piece for scaling AI-powered detection. However, a critical caveat must be addressed: security advisories have

highlighted significant vulnerabilities in older versions of Triton, including issues that could allow remote attackers to execute arbitrary code or cause denial-of-service conditions ^{62 63}. Therefore, any implementation using Triton must involve a rigorous policy of continuous monitoring for updates and meticulous patch management to mitigate these severe risks.

The final layer of the architecture encompasses the components responsible for orchestrating the entire workflow, correlating disparate alerts into meaningful incidents, and presenting actionable insights to security analysts. For orchestrating complex data pipelines, tools like Apache Airflow or Prefect can be employed ⁶. These are workflow orchestrators that schedule and manage the dependencies between different tasks (e.g., data extraction, model retraining, alert notification) but do not perform the heavy data processing themselves ⁶. For a purely free system, leveraging the native scheduling and state management features of a processing engine like Flink may suffice initially. Visualization and search capabilities are paramount for human operators. The Wazuh platform comes with its own Kibana-based dashboards, providing powerful tools for searching, analyzing, and visualizing security data collected by its agents ⁴⁵. Alternatively, general-purpose visualization tools like Grafana, often paired with a high-performance analytics database like ClickHouse, can be used to build custom, real-time dashboards tailored to specific security metrics and investigations ⁵. By integrating these components—Zeek/Suricata for data collection, Kafka/Flink for processing, Triton for serving, and Wazuh/Grafana for orchestration and visualization—a robust, end-to-end, and entirely free AI-based intrusion detection system can be architected.

Component Layer	Recommended Open-Source Tool(s)	Primary Function
Network Collection	Zeek (Bro), Suricata	Zeek provides deep network flow analysis and structured logging; Suricata acts as a rule-based NIDS/IPS generating alerts ^{31 38} .
Endpoint Monitoring	Wazuh, Sysmon	Wazuh serves as a HIDS and central manager, while Sysmon provides granular Windows event logging for enhanced visibility ^{11 38} .
Vulnerability Scanning	OpenVAS (Community Edition)	Provides proactive scanning for vulnerabilities in network assets, feeding findings into the detection pipeline ³⁸ .
Threat Intelligence	MISP (Malware Information Sharing Platform)	Gathers, shares, and correlates IOCs to enrich alerts and provide contextual threat data ^{16 18} .
Data Transport	Apache Kafka	Acts as a high-throughput distributed message broker to move data between different system components ^{2 6} .
Real-Time Processing	Apache Flink, Apache Spark Streaming	Processes data streams in real-time for complex event correlation, anomaly detection, and machine learning feature generation ^{2 6} .
AI Model Serving	NVIDIA Triton Inference Server	Deploys and serves trained AI models from various frameworks (e.g., PyTorch, TensorFlow) for low-latency inference ^{14 58} .
Orchestration & UI	Wazuh (with Kibana), Grafana, Superset	Wazuh provides a central hub for correlation and visualization; Grafana/Superset offer customizable dashboards for real-time analytics ^{5 45} .

This modular blueprint provides a strategic foundation for building a powerful and extensible cybersecurity platform. The selection of best-in-class free tools ensures that the system is not only cost-free but also benefits from the collective expertise and continuous improvement of vibrant open-source communities.

Data Strategy: Fueling AI Models with Publicly Available Cybersecurity Datasets

The efficacy of any AI-based Intrusion Detection System (IDS) is fundamentally determined by the quality, relevance, and volume of the data used to train and evaluate its predictive models. A robust data strategy is therefore not merely a supporting activity but a core pillar of the system's architecture. Under the constraint of being 100% free, this strategy relies exclusively on publicly available, open-source datasets that have become benchmarks within the cybersecurity research community. These datasets provide the necessary fuel for developing models capable of distinguishing between benign network activity and a wide array of malicious behaviors. The selection of appropriate datasets, coupled with a rigorous and standardized preprocessing pipeline, is critical to achieving high detection accuracy and minimizing false positives.

Several key datasets are frequently cited across the provided research materials as essential resources for training and benchmarking intrusion detection models. The **UNSW-NB15 dataset** is one of the most prominent and widely used ^{20 21}. Created by the Cyber Range Lab at UNSW Canberra, it was generated to simulate a realistic modern network environment by combining 100 GB of real-world user activities with synthetic contemporary attack behaviors ²⁰. The dataset contains nine distinct classes of attacks, including Fuzzers, Analysis, Backdoors, Denial of Service (DoS), Exploits, Generic, Reconnaissance, Shellcode, and Worms, providing a diverse and challenging testbed for classifiers ^{20 21}. It is available in multiple formats, including raw PCAP files, CSV files containing derived features, and logs from tools like Argus and Bro-IDS, offering flexibility for different research and development workflows ²⁰. Another cornerstone of modern IDS research is the **CIC-IDS2017 dataset**, created by the Canadian Institute for Cybersecurity (CIC) ^{21 32}. This dataset is notable for its focus on realistic benign traffic, generated by profiling normal user activities, which makes the embedded attacks more difficult to detect ²¹. It includes a variety of contemporary threats such as Brute Force FTP/SSH, DoS, Heartbleed, Web Attacks, Botnet, and DDoS scenarios ²¹. The follow-up **CIC-IDS2018 dataset** further expands this collection, featuring seven distinct attack scenarios gathered in a controlled environment, providing another valuable resource for model evaluation ²¹. For organizations with a focus on securing IoT and Industrial IoT (IIoT) environments, the **TON_IoT (The Other Network - Internet of Things) datasets** are indispensable ^{34 59}. Developed by the Cyber Range Lab at UNSW Canberra, these datasets were specifically designed to capture network traffic from simulated IoT and IIoT devices in a smart home environment, including both benign and malicious flows ^{51 59}. Using TON_IoT data is essential for ensuring that any developed IDS can operate effectively in resource-constrained edge settings ⁵⁵. Finally, for comparative studies and historical context, the **NSL-KDD dataset** remains a relevant benchmark, often used alongside newer datasets to evaluate the performance of novel algorithms ³⁵.

Simply acquiring these datasets is insufficient; their raw form is unsuitable for direct consumption by machine learning models. A thorough and methodical preprocessing pipeline is an absolute prerequisite. This pipeline addresses several critical tasks. First, it involves cleaning the data to handle missing values and inconsistencies ³². Second, it requires transforming all data into a numerical format that algorithms can process. This includes encoding categorical variables (e.g., converting protocol types from strings to numbers) and scaling features to ensure that no single feature dominates the model's learning process due to its scale ⁴⁴. One study demonstrated the importance of this step by implementing a pipeline that included feature scaling and binary encoding for categorical flags, which prepared the data for effective model training ¹⁴. A particularly

crucial aspect of preprocessing for intrusion detection is addressing class imbalance. In real-world network traffic, malicious events are vastly outnumbered by benign ones. If a model is trained on such imbalanced data, it will likely become biased towards predicting the majority class (benign), rendering it useless for detection. To combat this, researchers employ techniques like Random Oversampling (RO), which artificially increases the number of instances in the minority class to create a more balanced dataset for training ²¹. The impact of these preprocessing steps cannot be overstated. One study reported that by implementing a rigorous pipeline involving RO, Stacking Feature Embedding, and Principal Component Analysis (PCA), they achieved near-perfect accuracy on benchmark datasets using standard classifiers like Random Forest and XGBoost ²¹.

The table below summarizes the key characteristics of the most frequently referenced public datasets, highlighting their unique contributions to the field of intrusion detection research.

Dataset Name	Key Characteristics	Attack Types Included	Relevance for Modern IDS
UNSW-NB15	Combination of real user activity and synthetic modern attacks; 100GB of raw traffic captured ²⁰ .	Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms ²¹ .	High; Provides a mix of realism and syntheticity, serving as a classic benchmark for evaluating classifier performance.
CIC-IDS2017	Focus on realistic benign traffic generation by profiling user activities; includes modern web and brute-force attacks ²¹ .	Brute Force FTP/SSH, DoS, Heartbleed, Web Attack, Botnet, DDoS ²¹ .	Very High; Considered a benchmark for evaluating models against contemporary, realistic threats.
CIC-IDS2018	Collaborative dataset from CSE and CIC; seven distinct attack scenarios in a controlled environment ²¹ .	Brute-force, Heartbleed, Botnet, Denial of Service, Distributed Denial of Service, Web attacks, network infiltration ²¹ .	High; Offers a fresh dataset for validation and helps prevent overfitting to older benchmarks.
TON_IoT	Specifically designed for IoT/IIoT environments; captures traffic from simulated smart home devices ^{34 59} .	Labeled flows of both benign and malicious activity specific to constrained, embedded device networks ⁵¹ .	Critical for IoT/Edge deployments; essential for testing the viability of an IDS in resource-constrained environments.
NSL-KDD	Predecessor to UNSW-NB15; widely used for historical comparisons and foundational algorithm testing ³⁵ .	Classic attack types from KDD Cup '99 (e.g., DoS, U2R, R2L).	Moderate; Primarily useful for comparing new models against established baselines, though considered outdated for modern threats.

Beyond dataset selection and preprocessing, the machine learning models themselves play a pivotal role. Various algorithms have been successfully applied to these datasets. Studies consistently show high performance from ensemble methods like **Random Forest (RF)**, **Extra Trees (ET)**, and **XGBoost (XGB)** ²¹. For instance, one study reported accuracies of 99.59% for RF and ET on the UNSW-NB15 dataset in a binary classification task ²¹. Another paper proposed a novel model using XGBoost as its primary classifier,

integrated into the Nvidia Morpheus framework for high-speed network traffic analysis ¹⁴. Neural network-based approaches, such as Deep Neural Networks (DNN) and Multi-Layer Perceptrons (MLP), have also been evaluated, often in conjunction with explainability frameworks ^{35 36}. The choice of algorithm often depends on the specific requirements of the system, such as the need for interpretability versus raw predictive power. For example, Decision Trees (DT) are inherently interpretable, whereas DNNs are often treated as black-box models requiring external explainability techniques ³⁶. The computational complexity of these models is also a factor; for instance, the time complexity of a Decision Tree is $O(n*m)$, where n is the number of data points and m is the number of features, while a Random Forest with k trees has a complexity of $O(k*n*m)$ ²¹. This highlights the need to balance model performance with the available computational resources, especially in edge or IoT deployments. Ultimately, a successful data strategy combines access to high-quality, diverse, and up-to-date public datasets with a disciplined and scientifically sound preprocessing pipeline and a thoughtful selection of appropriate machine learning algorithms.

Implementation Framework: Deployment, Scalability, and Operationalization

Translating the architectural blueprint and data strategy into a functional system requires a pragmatic implementation framework that addresses the significant challenges of deployment, scalability, and ongoing operationalization. Building a complex, multi-component security stack from open-source tools introduces substantial operational overhead and a steep learning curve. Success hinges on a phased rollout strategy, careful infrastructure planning, and a deep understanding of the resource requirements and interoperability of each component. The goal is to create a system that is not only powerful but also maintainable and adaptable to the specific needs of the organization, whether it be a small office, a large enterprise, or a distributed cloud environment.

One of the foremost practical challenges is managing the resource footprint of the chosen tools. Many powerful open-source security platforms are not lightweight. For example, Wazuh's performance scales directly with the number of monitored agents; a deployment at the 1,000-agent scale can require a distributed infrastructure totaling 16 to 24 CPU cores and 32 to 48 GB of RAM across its nodes ²⁷. Similarly, Elasticsearch, a common component in many SIEM stacks, is known to be memory-intensive, often requiring a minimum of 16 GB of RAM to function effectively ⁴⁵. This reality necessitates a tiered

and strategic deployment plan. For resource-constrained environments, such as those involving IoT devices or remote branch offices, a centralized processing model is often impractical. Here, the concept of edge computing becomes critical. Deploying lightweight Kubernetes distributions like **K3s** is a key enabler for this strategy ²². K3s is a certified Kubernetes distribution designed specifically for production workloads in unattended, resource-constrained locations, such as Raspberry Pi clusters or edge servers ^{22 24}. By running data collection agents and even some filtering logic on these edge devices, it becomes possible to aggregate and summarize telemetry locally before forwarding only the most relevant data to a central server, drastically reducing bandwidth usage and egress costs ⁶. For larger-scale deployments, cloud infrastructure provides a flexible and scalable hosting solution. Several major cloud providers, including Google Cloud Platform (GCP) and Oracle Cloud Infrastructure (OCI), offer "always free" tiers that can host the core components of the security stack ^{23 47}. For instance, the OCI always free tier provides enough resources (2 vCPUs and 12GB of memory) to provision a functional Kubernetes cluster using k3s, which can serve as the foundation for the entire system ^{47 50}. This allows organizations to implement a powerful, free security monitoring solution with no initial financial outlay for infrastructure.

Integration and interoperability represent another major hurdle. A system built from disparate open-source tools must overcome the challenge of data silos and manual processes. The success of the proposed architecture depends on seamless communication between components. A prime example of this is the integration between Wazuh, Zeek, and Suricata ^{37 38}. In this setup, Zeek is configured to write its structured JSON logs to a specific directory. The Wazuh manager is then configured to monitor this directory, and a custom decoder is implemented to parse the Zeek logs into a format that Wazuh can understand and correlate with alerts from its own HIDS agents ³⁸. This allows for powerful cross-domain correlations, such as identifying when an internal host that was previously flagged by Wazuh for suspicious activity later connects to an IP address that Zeek's DNS logs identified as a newly registered domain associated with malware ³⁸. Similarly, integrating Suricata alerts into Wazuh follows a similar pattern, providing a unified view of both network-level rule-matches and host-level anomalies ²⁸. This principle of integration extends to threat intelligence platforms like MISP, which can be configured to automatically feed indicators of compromise (IOCs) into Wazuh to enrich local alerts ¹⁶. The ability to automate these integrations through configuration files, decoders, and APIs is what transforms a collection of standalone tools into a cohesive, intelligent security platform.

The following table outlines a phased implementation roadmap, inspired by a recommended deployment sequence for a free security stack, which can guide organizations from initial setup to a mature, fully integrated system 38 .

Phase	Duration	Key Activities	Core Components Deployed	Rationale
Phase 1: Foundational Visibility	Week 1	Deploy the central management server and install Wazuh agents on all endpoints (servers, desktops). Configure basic log collection and monitoring.	Wazuh Manager, Wazuh Agents	Establishes a baseline of endpoint visibility and creates a central hub for all subsequent components.
Phase 2: Enhanced Endpoint Telemetry	Week 2	Deploy Sysmon on Windows endpoints and configure it with a robust configuration file (e.g., SwiftOnSecurity). Ensure Wazuh agents are collecting Sysmon events.	Wazuh, Sysmon	Significantly improves the depth and granularity of endpoint data, enabling detection of more advanced threats like process hollowing or lateral movement.
Phase 3: Comprehensive Network Monitoring	Weeks 3-4	Deploy Zeek on a network tap or SPAN port to capture full network traffic. Integrate Zeek's JSON logs with the Wazuh manager. Deploy and integrate Suricata for rule-based network alerts.	Wazuh, Zeek, Suricata	Adds deep network visibility and real-time threat detection capabilities, allowing for correlation between endpoint and network events.
Phase 4: Proactive Threat Hunting	Week 5	Install and configure OpenVAS to perform regular vulnerability scans on network assets. Correlate scan results with active threats and asset inventory.	OpenVAS / Greenbone Community Edition	Shifts the security posture from reactive detection to proactive risk reduction by identifying and prioritizing exploitable weaknesses.
Phase 5: Automated Behavioral Blocking	Week 6	Deploy CrowdSec to read logs from services like SSH and web servers. Configure bouncers to automatically block malicious IPs based on detected attack patterns.	CrowdSec	Implements an automated IPS capability, blocking brute-force attacks and other repetitive malicious behaviors at the firewall level.
Phase 6: Deception and Advanced Analytics	Ongoing	Deploy Canary Tokens to create decoy assets. Begin developing and integrating custom AI/ML models for anomaly detection.	Canary Tokens, Custom ML Models	Adds layers of deception for early threat detection and begins leveraging the full power of AI for uncovering unknown threats.

Finally, the operationalization of the system involves establishing clear processes for managing the continuous flow of data and alerts. This includes setting up reliable data retention policies, ensuring the system is monitored for its own health, and training personnel to effectively use the tools. The sheer volume of data handled by a system built on Kafka and Flink requires careful management of storage and processing resources. The "hot/cold" data split strategy is a common architectural pattern, where recent, high-value "hot" data is stored in memory for fast querying, while older "cold" data is archived to cheaper, disk-based storage for long-term analysis and compliance purposes 4 . The operational complexity of tools like Apache Flink and Apache Kafka should not be underestimated; they require significant expertise to manage production clusters

effectively 2 6 . Therefore, starting with a simpler, Wazuh-centric architecture and gradually introducing more complex components like Flink aligns with the principle of incremental value delivery. By adopting a phased implementation framework, carefully planning for scalability, and mastering the art of integration, organizations can successfully navigate the challenges of building and maintaining a powerful, free, and open-source cybersecurity monitoring platform.

Operationalizing the System: From Alert Generation to Incident Response

An AI-based intrusion detection system, no matter how sophisticated, is only as valuable as its ability to translate raw data and model outputs into actionable security outcomes. Operationalizing the system involves designing and implementing workflows that efficiently manage the lifecycle of an alert, from its initial generation to its resolution. This process is centered on three key pillars: effective alert management to reduce noise and prioritize threats, structured incident response to coordinate containment and eradication efforts, and the crucial integration of human oversight to ensure decisions are sound and accountable. A purely automated system is premature for most security contexts; a hybrid model that leverages automation for speed and scale while retaining human judgment for complex decision-making is the most effective approach.

The first step in the operational workflow is alert management. In a high-volume environment, the output of an IDS can quickly become overwhelming, leading to analyst fatigue and missed critical threats. The goal of alert management is to filter, correlate, and triage alerts to surface the most important events for human review. **Keep** is an open-source AIOps and alert management platform specifically designed for this purpose 9 10 . Keep allows security teams to define declarative YAML files, known as "Workflows," which automate the entire alert lifecycle 9 . These workflows can automatically enrich an incoming alert with data from other sources (like MISP for threat intelligence), suppress known false positives, and route the alert to the appropriate analyst or team based on predefined rules. This automation significantly reduces manual toil and accelerates the time to first response. For more complex and collaborative incident handling, a dedicated Security Incident Response Platform (SIRP) is required. **TheHive** is a purpose-built open-source SIRP designed for Security Operations Centers (SOCs), Computer Security Incident Response Teams (CSIRTs), and CERTs 15 19 . TheHive provides a structured environment for managing the full incident lifecycle, from

initial alert triage to final reporting ¹⁹. When an alert from the IDS (e.g., Wazuh) is deemed worthy of investigation, it can be automatically or manually promoted into a case within TheHive. This case serves as a central repository for all related information, communications, and actions taken during the investigation.

To elevate the effectiveness of an incident case, the case can be enriched with automated analysis. This is where **Cortex** comes into play. Cortex is an open-source pluggable framework that integrates seamlessly with TheHive ¹⁶ ¹⁷. It provides a suite of built-in analyzers for performing deep-dive investigations on indicators of compromise (IoCs) extracted from the case, such as IP addresses, domain names, file hashes, and URLs ¹⁷. For example, when investigating a suspicious outbound connection, Cortex can automatically query VirusTotal for the destination IP, check a URLhaus database for the domain, and run the associated file through a sandboxing service. The results of these analyses are then attached directly to the TheHive case, providing investigators with a wealth of contextual data without leaving their primary tool. This tight integration between TheHive for case management and Cortex for automated analysis creates a powerful and efficient SOC toolkit. The entire stack can be further enhanced by integrating **MISP** (Malware Information Sharing Platform), which can be used to share threat intelligence and correlate IoCs across different teams or even with external partners, closing the loop on threat information ¹⁶ ¹⁸.

A critical element underpinning effective incident response is the Human-In-The-Loop (HITL) mechanism. Full automation in security, especially concerning remediation actions, carries significant risk. A HITL workflow ensures that a human expert can intervene, validate, and authorize critical actions, preventing catastrophic errors. Several open-source frameworks and platforms explicitly support this principle. The Keep platform supports HITL by allowing workflows to pause and wait for an external signal, such as a webhook call or input from a web form, before proceeding to the next step ⁶⁰. This allows an analyst to review a proposed action (e.g., "Block this IP and quarantine this host") and either approve or reject it. The cybersecurity automation framework **CAI** (Cybersecurity AI) is built around a HITL philosophy from the ground up ¹³. Its agent-centric architecture allows users to interrupt any automated process at any time by pressing Ctrl+C to provide guidance, correct course, or supply new instructions. This interactive control is vital for navigating the unpredictable nature of live security operations. Specialized tools like the **Agent User Interaction Protocol (AG-UI)** tool system are also designed specifically to facilitate HITL workflows by defining tools that explicitly request human input during a process ⁵⁷. This principle of shared control is not just a safety feature; it is a fundamental requirement for building trust in AI-driven

systems and ensuring that they augment, rather than replace, the irreplaceable expertise of a human security analyst.

The operational workflow, therefore, can be conceptualized as a journey from detection to resolution: 1. **Detection:** The AI models running on the stream processing engine (e.g., Flink) analyze real-time data and generate an alert, which is sent to the central management platform (e.g., Wazuh). 2. **Alert Management & Triage:** The alert is ingested by an AIOps platform like Keep. It is automatically enriched with threat intelligence from MISP and correlated with other events. Based on severity and confidence, it is escalated to a human analyst or automatically promoted to a case in TheHive. 3. **Investigation:** Within TheHive, the incident responder opens the case. They use Cortex to launch automated queries against various threat intelligence and analysis services, populating the case with valuable context about the IoCs involved. 4. **Human-in-the-Loop Decision:** The analyst reviews all the gathered evidence. They might decide to take a remediation action, such as blocking an IP address. This action is initiated through a HITL-enabled workflow in Keep or directly within TheHive, which pauses to await explicit approval from the analyst. 5. **Remediation & Containment:** Upon approval, the action is executed. This could involve sending a command to a Wazuh agent to disable a user account, or triggering a CrowdSec bouncer to add the malicious IP to a firewall blocklist. 6. **Reporting & Closure:** Once the threat is contained and eradicated, the analyst documents their findings and the actions taken within the TheHive case. The case is then closed, and lessons learned are captured for future improvement.

By embedding these principles of intelligent alert management, structured incident response, and mandatory human oversight into the system's design, an organization can transform its AI-driven detection capabilities into a tangible and effective security operation. This hybrid human-machine approach is the key to responsibly harnessing the power of AI in cybersecurity.

Advanced Enhancements: Evolving the IDS with Zero-Cost Innovations

While a baseline AI-based Intrusion Detection System can be constructed using the foundational open-source components discussed, its long-term effectiveness and adaptability depend on continuous evolution. The landscape of cyber threats is dynamic, and a static detection model will inevitably fall behind. Fortunately, the open-source

community is a hotbed of innovation, producing numerous free frameworks and methodologies that can significantly enhance the capabilities of a security platform. This section explores three promising, zero-cost enhancements that can elevate a standard IDS: the integration of Explainable AI (XAI) for transparency and trust, the adoption of Federated Learning (FL) for privacy-preserving collaborative defense, and the development of Lightweight Edge AI models for securing constrained IoT and industrial environments.

Explainable AI (XAI) addresses one of the most significant barriers to the adoption of machine learning in security: the "black box" problem. Many powerful models, such as deep neural networks, produce highly accurate predictions but offer little insight into the reasoning behind them. When an XGBoost or DNN model flags a legitimate business transaction as fraudulent, an analyst is left with no way to understand *why*, making it difficult to trust the system or tune it to reduce false positives. XAI provides a set of tools and techniques to make these model decisions transparent and interpretable. Several open-source frameworks have been developed to integrate XAI capabilities directly into IDS pipelines. The **XAI-IDS** framework is a notable example, providing a public baseline for the community ³⁵. It integrates two of the most popular open-source XAI libraries, SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations), to generate both global and local explanations ³⁵. Global explanations, such as SHAP summary plots, show which features (e.g., packet size, TCP flags, payload entropy) are most influential in the model's overall decision-making process. Local explanations, generated by LIME, provide insight into why a *specific* data point was classified as malicious by highlighting the exact features that contributed most to that decision ³⁵. Another framework, **L-XAIDS**, similarly uses LIME for local explanations and ELI5 for global feature importance rankings, demonstrating high accuracy (85%) on the UNSW-NB15 dataset while providing these crucial interpretability features ³⁶. The practical value of XAI is immense. By understanding that an alert was triggered because of "a high number of failed login attempts followed by a connection to a Tor exit node," an analyst can validate the alert much faster and gain confidence in the model's ability to detect credential stuffing attacks. This transparency fosters trust, reduces the time spent on false positives, and empowers analysts to collaborate with data scientists to refine and improve the underlying models.

Federated Learning (FL) offers a revolutionary approach to collaborative threat detection that respects data privacy and sovereignty. In many organizations, security data is siloed—for example, different branches of a company, separate departments, or partner organizations may all possess valuable security logs, but sharing this raw data is prohibited by regulations or corporate policy. FL allows these parties to collaboratively train a single, powerful, shared global model without ever exposing their sensitive,

localized data ⁴¹. The process works by sending the current model parameters to each participating client. Each client then trains the model on its local dataset and sends back only the updated model weights (not the data) to a central parameter server. This server aggregates the updates to create a new, improved global model, which is then redistributed to all clients ⁴¹. This cycle repeats until the model converges. The open-source landscape for FL is rapidly maturing, with several production-ready frameworks available under permissive licenses like Apache 2.0. **Flower** stands out as a highly flexible, framework-agnostic option that works with popular ML libraries like PyTorch and TensorFlow, making it easy to integrate into existing workflows ^{39 41}. **FATE (Federated AI Technology Enabler)**, backed by WeBank, is another production-ready framework that supports both horizontal and vertical data partitioning and implements advanced security measures like secure multi-party computation (MPC) protocols for encryption ⁴⁰. However, implementing a robust FL system is non-trivial. Production readiness varies; TensorFlow Federated (TFF) currently lacks a true federated mode, while FATE has significant hardware requirements (6 GB RAM, 100 GB disk space) and a steep learning curve ⁴⁰. Despite these challenges, FL represents a powerful long-term enhancement. A future research direction for an IDS could involve extending an XAI framework into a decentralized version using a federated learning approach, enabling knowledge sharing across different network nodes while maintaining strict data privacy ³⁶.

The proliferation of IoT and Industrial IoT (IIoT) devices introduces a new frontier for intrusion detection, characterized by extreme resource constraints. These devices often have limited processing power, memory, and battery life, making it impossible to run conventional security software or complex AI models. This has spurred a wave of research focused on designing and deploying **Lightweight Edge AI Models** specifically for these environments ⁵⁶. The goal is to create models that are not only accurate but also computationally frugal. Recent work in this area demonstrates significant progress. One study focused on the design, training, and deployment of lightweight AI models for IoT network intrusion detection at the edge, highlighting the need for efficiency ⁵⁶. Another paper proposed a resource-efficient model that achieved a remarkable latency of just 1.8 milliseconds on a Raspberry Pi 4 (a device with a 1.5 GHz CPU and 2 GB of RAM) while consuming only about 10 millijoules of energy ⁶⁴. This level of performance on commodity edge hardware proves that sophisticated AI-driven detection is feasible even in the most constrained settings. For an organization looking to build a truly comprehensive free security system, this line of research suggests that simply using free tools may not be enough for IoT protection. It may be necessary to engage in a degree of custom development, fine-tuning existing open-source models or training new ones specifically optimized for the target edge hardware. The EdgeX Foundry project, an open-

source edge platform, provides a framework for building applications that can run on edge devices and communicate with cloud systems, potentially serving as a foundation for deploying these lightweight models ⁴³.

In summary, the path to a truly advanced, free IDS involves moving beyond a simple aggregation of tools. By strategically incorporating zero-cost innovations like XAI for transparency, Federated Learning for collaborative defense, and Lightweight Edge AI for IoT security, an organization can build a system that is not only reactive but also adaptive, trustworthy, and future-proof. These enhancements, drawn from cutting-edge open-source research, provide a clear roadmap for evolving the system to meet the challenges of tomorrow's threat landscape.

Enhancement Area	Description	Key Open-Source Frameworks/ Tools	Potential Impact
Explainable AI (XAI)	Techniques to make the decisions of "black box" AI models transparent and understandable to human analysts.	XAI-IDS (using SHAP, LIME), L-XAIDS (using LIME, ELI5) ^{35 36} .	Increases analyst trust, reduces false positives, and enables model refinement based on clear feedback.
Federated Learning (FL)	A collaborative machine learning approach that trains a shared global model across multiple decentralized devices/ organizations without centralizing raw data.	Flower, FATE, NVIDIA FLARE, TensorFlow Federated (TFF) ^{39 40} .	Enables privacy-preserving threat intelligence sharing between different entities (e.g., company branches, partners).
Lightweight Edge AI	Designing and deploying highly efficient AI models optimized to run on resource-constrained IoT and edge devices.	Research-focused; involves custom model design (e.g., pruning, quantization) and frameworks like EdgeX Foundry ^{43 56} .	Extends advanced AI-based intrusion detection capabilities to IoT and Industrial IoT (IIoT) environments.

Synthesis and Strategic Recommendations

This deep research investigation confirms that the design and development of a fully free, AI-based intrusion detection and cybersecurity monitoring system is a challenging yet entirely achievable objective. The analysis of available open-source tools, datasets, and methodologies reveals a clear path toward constructing a powerful and extensible security platform. The central conclusion is that success is contingent not on finding a single, magical tool, but on the deliberate and strategic integration of a curated stack of specialized, best-in-class free components into a modular, layered architecture. Such a system can provide capabilities rivaling commercial solutions, all while operating under the critical constraint of zero cost.

The recommended architectural blueprint consists of several distinct but interconnected layers. At the base, a data collection layer comprising **Zeek** for deep network analysis, **Suricata** for rule-based network intrusion detection, **Wazuh** for comprehensive endpoint monitoring (augmented with **Sysmon** for Windows), and **OpenVAS** for proactive vulnerability scanning, provides a rich and multi-faceted source of security telemetry. This raw data is transported via **Apache Kafka** to a powerful real-time stream processing engine, with **Apache Flink** being the optimal choice for its low-latency, stateful processing capabilities essential for advanced threat detection. The processed data fuels AI models, which are deployed and served efficiently using **NVIDIA Triton Inference Server**. All findings are aggregated, correlated, and visualized through the **Wazuh** platform, which acts as the central nervous system of the entire operation. For advanced workflows, this core can be extended with a Security Incident Response Platform (**TheHive**) and an automated analysis framework (**Cortex**) for structured incident management, and an alert management platform (**Keep**) for intelligent automation.

However, the success of this architecture is critically dependent on a robust data strategy. The system's AI models must be fueled by high-quality, relevant, and up-to-date data. This requires leveraging publicly available, free cybersecurity datasets such as **CIC-IDS2017**, **TON_IoT**, and **UNSW-NB15**, and subjecting them to a rigorous preprocessing pipeline that includes normalization, encoding, and addressing class imbalance. Without this scientific approach to data, even the most sophisticated models will fail. Furthermore, the implementation must be pragmatic. Recognizing the significant resource requirements and operational complexity, a phased rollout is strongly advised. Starting with a Wazuh-centric architecture to establish foundational visibility and gradually introducing more complex components like Flink and Triton allows for incremental value delivery and skill-building. For resource-constrained environments like IoT, leveraging lightweight container orchestrators like **K3s** is essential for deploying data collection agents at the edge.

Operationally, the system must be designed as a human-machine partnership. While automation through platforms like Keep can accelerate response times, a mandatory Human-In-The-Loop (HITL) is non-negotiable for authorizing critical remediation actions. This hybrid approach balances the speed and scale of machines with the nuanced judgment of human experts, mitigating the risks of autonomous decision-making. Finally, to ensure the system remains effective against future threats, it must be built with extensibility in mind. The integration of zero-cost innovations is the key to long-term relevance. Prioritizing the implementation of **Explainable AI (XAI)** using frameworks like XAI-IDS provides immediate value by increasing analyst trust and reducing false positives. Positioning **Federated Learning (FL)** as a future goal can enable privacy-

preserving collaborative defense, while researching **Lightweight Edge AI** is crucial for securing the burgeoning IoT landscape.

In executing this vision, it is imperative to remain vigilant about the security of the toolchain itself. The very tools chosen to defend the network may contain vulnerabilities. A diligent policy of continuous security auditing and patch management, particularly for components with known risks like older versions of NVIDIA Triton, is an essential part of the operational posture ⁶² ⁶³. By adhering to this synthesized blueprint—a modular architecture, a data-driven approach, a pragmatic implementation plan, a human-centric operational workflow, and a commitment to continuous innovation—an organization can successfully build and operate a sovereign, powerful, and cost-free cybersecurity monitoring system capable of meeting the demands of the modern threat landscape.

Reference

1. Stream processing guide: Real-time data and use cases - Fivetran <https://www.fivetran.com/learn/stream-processing>
2. 8 Best Data Streaming Tools in 2026: Compared & Ranked - Mimacom <https://www.mimacom.com/learning-hub/data-streaming-tools>
3. Data Streaming Platforms for Real-Time Analytics & Integration <https://www.striim.com/blog/data-streaming-platforms-for-real-time-analytics-and-integration/>
4. How to do real-time data processing for modern analytics 2026 - Tinybird <https://www.tinybird.co/blog/real-time-data-processing>
5. Advice need on open-source real-time data streaming platform https://www.reddit.com/r/dataengineering/comments/1dikvtd/advice_need_on_opensource_realtime_data_streaming/
6. Data Pipeline Tools: 8 Platforms Compared for 2026 | Expanso <https://expanso.io/blog/data-pipeline-tools/>
7. 9 Best Open Source Tools for Stream Processing - simplyblock <https://simplyblock.io/blog/open-source-tools-for-stream-processing/>
8. A look at 8 top stream processing platforms - Aply Realtime <https://ably.com/blog/a-look-at-8-top-stream-processing-platforms>
9. keephq/keep: The open-source AIOps and alert ... <https://github.com/keephq/keep>

10. Keep - Open-source AIOps platform <https://www.KeepHQ.dev/>
11. Awesome Security - GitHub <https://github.com/sbilly/awesome-security>
12. Awesome AI in Cybersecurity - GitHub <https://github.com/ElNiak/awesome-ai-cybersecurity>
13. aliasrobotics/cai: Cybersecurity AI (CAI), the framework for ... <https://github.com/aliasrobotics/CAI>
14. A Modular AI-Driven Intrusion Detection System for ... <https://www.mdpi.com/1424-8220/25/1/130>
15. The Hive, Cortex and MISP install (Docker) - Medium <https://mahimsec.medium.com/setting-up-the-incident-response-platform-the-hive-cortex-and-misp-install-docker-ad98dca26e13>
16. How to Integrate Cortex & MISP with TheHive in your SOC https://www.youtube.com/watch?v=ovUuNQsW_FQ
17. Threat Hunting with The Hive, Cortex and MISP - CTIN <https://cyberthreatintelligencenetwork.com/index.php/2023/03/30/threat-hunting-with-thehive-cortex-and-misp/>
18. About MISP Integration - TheHive 5 Documentation <https://docs.strangebee.com/thehive/administration/misp-integration/about-misp-integration/>
19. TheHive <https://strangebee.com/thehive/>
20. The UNSW-NB15 Dataset <https://research.unsw.edu.au/projects/unswnb15dataset>
21. Machine learning-based network intrusion detection for big and ... <https://arxiv.org/html/2401.12262v1>
22. K3s <https://k3s.io/>
23. k3s on GCP's always free tier - cwiggs <https://cwiggss.com/posts/2019-06-19-k3s-on-gcp-always-free-tier/>
24. Let's Make a k3s Cluster with Raspberry Pi and Cloudflare Tunnel https://dev.to/hiro_111/lets-make-a-k3s-cluster-with-raspberry-pi-and-cloudflare-tunnel-3l33
25. How to Deploy K3s on Edge Devices - OneUptime <https://oneuptime.com/blog/post/2026-01-25-deploy-k3s-edge-devices/view>
26. A weekend project with K3s and Oracle Cloud free tier. - Medium <https://medium.com/@Phoenixforge/a-weekend-project-with-k3s-and-oracle-cloud-free-tier-99eda1aa49a0>
27. Wazuh vs Splunk: \$0 vs \$300K/yr SIEM [2026] - shattered.io <https://shattered.io/wazuh-vs-splunk/>
28. Network IDS integration - Proof of Concept guide <https://documentation.wazuh.com/current/proof-of-concept-guide/integrate-network-ids-suricata.html>

29. Network security monitoring with Wazuh and Zeek <https://wazuh.com/blog/network-security-monitoring-with-wazuh-and-zeek/>
30. Network IDS Integration with Wazuh and Suricata - Medium <https://mahimsec.medium.com/network-ids-integration-with-wazuh-and-suricata-f2177b848dbe>
31. Suricata Integration with Wazuh [DL Series-6] - LinkedIn <https://www.linkedin.com/pulse/suricata-integration-wazuh-dl-series-6-gibin-k-john-kwofc>
32. CICIDS2017 - Comprehensive Data Processing for ML <https://www.kaggle.com/code/ericnacletoribeiro/cicids2017-comprehensive-data-processing-for-ml>
33. Intrusion detection evaluation dataset (CIC-IDS2017) <https://www.unb.ca/cic/datasets/ids-2017.html>
34. The TON_IoT Datasets - UNSW Research <https://research.unsw.edu.au/projects/toniot-datasets>
35. ogarreche/XAI_NIDS: Towards an Explainable Artificial Intelligence ... https://github.com/ogarreche/XAI_NIDS
36. L-XAIDS: A LIME-based eXplainable AI framework for Intrusion Detection ... <https://arxiv.org/html/2508.17244v1>
37. Integrate Wazuh and Suricata for real-time network threat detection <https://www.facebook.com/groups/564863590763290/posts/2161343937781906/>
38. Zero-Budget Security Stack 2026: Wazuh, Suricata, OpenVAS <https://www.decryptiondigest.com/blog/zero-budget-security-stack-free-tools>
39. Top 7 Open-Source Frameworks for Federated Learning <https://www.apheris.com/resources/blog/top-7-open-source-frameworks-for-federated-learning>
40. Open-Source Federated Learning Frameworks for IoT <https://www.mdpi.com/1424-8220/21/1/167>
41. Design a federated learning system in seven steps <https://openmined.org/blog/design-a-federated-learning-system-in-seven-steps/>
42. Building AI-Driven Detection Pipelines to Counter AI- ... <https://arctiq.com/blog/building-ai-driven-detection-pipelines-to-counter-ai-powered-malware>
43. EdgeX Foundry, The Open Source Edge Platform <https://www.edgexfoundry.org/platform-old/>
44. Intrusion Detection System [NSL-KDD] <https://www.kaggle.com/code/eneskosar19/intrusion-detection-system-nsf-kdd>
45. Wazuh Alternatives: The Best Open-Source SIEM Options <https://obsidiancorps.com/en/blog/2026/06/wazuh-alternatives-open-source>
46. Getting started with Wazuh and Suricata - Packt <https://www.packtpub.com/en-us/product/security-monitoring-with-wazuh-9781837632152/chapter/chapter-1->

intrusion-detection-system-ids-using-wazuh-2/section/getting-started-with-wazuh-and-suricata-ch02lv11sec05?srsltid=AfmBOor-MS1Rd--6gisgyE_Jmt75DVWvs5GzOM9tl0yHKgLd-Xpag-vv

47. Deploy a Kubernetes Cluster (k3s) on Oracle Always Free ... https://www.reddit.com/r/oraclecloud/comments/1qabcs9/deploy_a_kubernetes_cluster_k3s_on_oracle_always/
48. Deploy a Kubernetes cluster for free, using k3s and Oracle ... <https://garutilorenzo.github.io/deploy-kubernetes-for-free-oracle-cloud/>
49. Getting started with Wazuh and Suricata - Packt https://www.packtpub.com/en-us/product/security-monitoring-with-wazuh-9781837632152/chapter/chapter-1-intrusion-detection-system-ids-using-wazuh-2/section/getting-started-with-wazuh-and-suricata-ch02lv11sec05?srsltid=AfmBOopZVmAAL_X6K6QZUF2NaozNiiZk4QdBJEnEy4Ph1qrHLMJb16cN
50. nce/oci-free-cloud-k8s: always free kubernetes on oracle cloud · GitHub <https://github.com/nce/oci-free-cloud-k8s>
51. TON_IoT Network Dataset <https://www.kaggle.com/datasets/arnobbhowmik/ton-iot-network-dataset>
52. sunglasses: A curated list of awesome MLOps tools <https://github.com/kelvins/awesome-mlops>
53. 15 Best Open-Source MLOps Tools for 2026 <https://www.cake.ai/blog/best-open-source-mlops-tools>
54. Enhancing Network Security by Integrating Suricata IDS ... <https://medium.com/@anshumaansingh10jan/enhancing-network-security-by-integrating-suricata-ids-with-wazuh-for-threat-detection-b6d5c6eeb9a0>
55. (PDF) A resource efficient IoT intrusion detection model ... https://www.researchgate.net/publication/411622970_A_resource_efficient_IoT_intrusion_detection_model_using_hybrid_feature_selection_for_edge_computing
56. Design and Deployment of Lightweight Edge AI Models ... <https://www.techrxiv.org/doi/10.36227/techrxiv.176184835.56908681>
57. Tools - Agent User Interaction Protocol <https://docs.ag-ui.com/concepts/tools>
58. Raj Balabantaray's Post https://www.linkedin.com/posts/raj-balabantaray-05822519_tritoninferenceserver-nvidia-modelserving-activity-7410597838272827392-ohKb
59. TON_IoT Network Dataset - UNSW Cyber Range Lab Collection <https://iotdataset.com/data/ton-iot-network-dataset-unsw>
60. v2.0 Breaking changes | Changelog <https://docs.n8n.io/changelog/v20-breaking-changes>

61. Deploying ML Models using Nvidia Triton Inference Server <https://medium.com/@siddharthashrestha/deploying-ml-models-using-nvidia-triton-inference-server-part-i-3d73be585dcd>
62. NVIDIA Triton Bugs Let Unauthenticated Attackers Execute ... <https://thehackernews.com/2025/08/nvidia-triton-bugs-let-unauthenticated.html>
63. Nvidia修補AI推論伺服器Triton高風險DoS漏洞 <https://www.ithome.com.tw/news/172727>
64. [PDF] robust and explainable hybrid deep learning model for real-time ... <https://thesesjournal.com/index.php/1/article/download/899/710>